

Speech Recognition - Project for Master students

Creating and testing an HMM PoS Tagger for alternative languages

Prof. Dr.-Ing. Udo Garmann

Introduction

Important: The project must be executed and submitted as a single student alone!

PoS-Tagging and HMMs will be handled in later lectures at the end of the course NLP. But introductions to these topics can be found in NLTK and the Jurafsky book (see https://web.stanford.edu/~jurafsky/slp3/ed3book_Jan25.pdf). There are also several tutorials about Part-of-Speech- (PoS) -tagging with Hidden-Markov-Models (HMM) available on the Web. An example is <https://wisdomml.in/hidden-markov-model-hmm-in-nlp-python/> (↗ visited 24/04/05).

So start with making yourself familiar with the terms PoS-tagging and HMM!

The project is about creating a PoS-tagger for a language other than English or German (called alternative language here).

Part 01: First steps

Find a PoS-tagged corpus for the alternative language, e.g. your native language. Take care that its texts can be read by a Python program (so it must be Unicode or better UTF-8).

Alternatively, if no tagged corpus can be found, you may do the following: Create an own tagged corpus for the alternative language. Select a text, that uses the language. Then PoS-tag the words in it. The corpus must include minimum 30 sentences (the more the better). For creating a corpus, an AI tool like ChatGPT may be used.

Use the same data structure (list of tuples) as described in the NLTK (see <https://www.nltk.org/book/ch05.html> , ↗ visited 24/04/05) .

Write a documentation about the language and the corpus. The documentation must be included in a README-file (see part 03). It must include definitions of:

- What is a sentence?
- What is a word token?
- What are typical Parts of Speech? Use at least the Universal Part-of-Speech Tagset described in the NLTK, chapter 5.
- Optionally, you may extend it for the used language. As a guideline, it can be similar to the Penn Treebank PoS Tags (see Jurafsky book 3rd edition draft, chapter 17.2), which is especially for English.

Important: Also provide an english translation of the used corpus.

Part 02: Setting up an HMM model with Python

Next, with Python create your own HMM model for PoS-tagging of the alternative language.

Train it with your corpus and evaluate it with at least 5 sentences.

Basic requests for the Python application are:

- runs from the command line (no GUI allowed)
- uses an up-to-date Python (≥ 3.10)
- no AI- or other online-APIs are allowed

Further requests are:

- evaluate tagged corpus,
- split corpus into train and test sentences,
- create HMM PoS-tagger,
- show accuracy of the PoS-tagger,
- read another test,
- output PoS-tags as list of tuples

Document your results in the README (see next part).

You may use ChatGPT or a similar tool in the project, but please document it in the README.

Part 03: Submission of the project

The projects' files must be archived in 1 ZIP file. This has to be submitted via the relevant submission slot in iLearn. Take care of the deadline stated there (english "due", german "fällig").

The project files include:

- requirements.txt, which can be used for the Python modules installation (`pip install -r requirements.txt`)
- README.md (Markdown format!), that describes the project. The format of the README must have the structure that is described later. (see further)
- Python files of the project
- additional files necessary for the Python app (created corpus, test data, ...)

Project grading:

- Correct Submission (20%)
- Implementation of the requests (30%). Next to the Python requirements (described above) the general requirements include:
 - description of sentences, words, and other relevant elements in the text exists,
 - description of the PoS-tags with examples
- Impression / completeness (30%)
- Implementation (20%)

Overall course grading for master students:

- Written exam 80%
- Project 20%, but prerequisite for passing the course

Format of the README.md

The README.md file must be written in English, use the Markdown format, and have the following structure:

The first line must have information about the student and look like this: lastname, firstname, mat-no

an empty line must be included

The title name of the project

an empty line must be included

A MD chapter called "Project description". In this chapter you describe the basics of the project including title and topic.

A MD chapter "Prerequisites", which describes the software prerequisites of the project. Especially declare the used version of Python here.

A MD chapter "Installation", which describes the installation of the project. Typically there is a basic installation and there may be additional steps like data loading.

A MD chapter "Basic Usage". It describes how to start the project, how to load a corpus and how to interpret an output.

A MD chapter "Alternative Language". It describes the used language with

- What is a sentence?
- Used PoS-tags with examples!

A MD chapter "Used AI"

If an AI like ChatGPT was used in the project, describe the purpose and the prompts here.

Later, a MD chapter "Implementation of the Requests" must be added. It describes what part of the code implements a request of the project. See the description of the requests in the NextCloud material folder and describe how you have implemented them.

README Example

```
Mustermann, Max, 123456
```

```
PoS-Tagger for Russian Language
```

```
# Project description
```

```
A PoS-Tagger was created ...
```

```
# Prerequisites
```

```
Python >= 3.10
```

```
...
```

```
# Installation
```

```
# Basic Usage
```

```
# Alternative Language
```

```
Russian language is similar other european languages, east slawic?, see  
  https://en.wikipedia.org/wiki/Russian_language (\ExternalLink, visited 24/04/05)
```

```
Graphemes different ("cyrillic")
```

```
# Used AI
```

```
ChatGPT was used to extend the training set.
```

```
Prompts:
```

- ```
- Print 30 sentences in modern Russian.
- ...
```